

Hibernate ID Generation Strategies

In Hibernate/JPA, `@GeneratedValue` is used to automatically generate values for a Primary Key.

Example:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Here:

- `@Id` → identifies the field as the **Primary Key**.
- `@GeneratedValue` → tells Hibernate that the ID should be generated automatically.
- `GenerationType` → specifies **how** the ID should be generated.

There are four main strategies:

- `GenerationType.AUTO`
- `GenerationType.IDENTITY`
- `GenerationType.SEQUENCE`
- `GenerationType.TABLE`

1. `GenerationType.AUTO`

`AUTO` means that Hibernate/JPA chooses an appropriate ID generation strategy based on the database, the ID type, and the configuration.

In simple words:

AUTO = Let Hibernate choose.

Example:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;
}
```

You don't specify whether Hibernate should use Identity, Sequence, or another mechanism. The provider decides.

When to use it?

Use AUTO when:

- You don't need to control the exact ID generation mechanism.
- You want a simple configuration.
- You want Hibernate to choose a suitable strategy.
- The application may work with different databases.

Example

When you save:

```
Student student = new Student();
student.setName("Ahmed");

studentRepository.save(student);
```

You don't need to set the ID manually.

Hibernate handles the ID generation.

Database

There is no specific database associated with AUTO.

The actual mechanism depends on the database and Hibernate configuration.

2. GenerationType.IDENTITY

What is it?

IDENTITY means that the **database generates the Primary Key** using an Identity or Auto Increment column.

In simple words:

IDENTITY = Database Auto Increment

Example:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

For MySQL, the table could be:

```
CREATE TABLE student (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100)
);
```

The important part is:

`AUTO_INCREMENT`

The database automatically generates the ID.

When to use it?

Use `IDENTITY` when your database supports Identity/Auto Increment columns.

It is commonly used with:

- MySQL
- SQL Server

Example

Suppose we insert three students:

Ahmed → ID = 1

Sara → ID = 2

Mohamed → ID = 3

The application does not generate these IDs. The database does.

Flow

Java Object



Hibernate



INSERT



Database generates ID

Database

Common examples:

MySQL

```
id BIGINT AUTO_INCREMENT PRIMARY KEY
```

SQL Server

```
id INT IDENTITY(1,1) PRIMARY KEY
```

3. GenerationType.SEQUENCE

What is it?

SEQUENCE uses a **Database Sequence** to generate Primary Key values.

A Sequence is a database object that generates numbers in sequence:

1 → 2 → 3 → 4 → 5 → ...

In simple words:

SEQUENCE = Database Sequence generates the ID

Example:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    private String name;
}
```

You can also explicitly configure the Sequence:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(
        strategy = GenerationType.SEQUENCE,
        generator = "student_seq"
    )
    @SequenceGenerator(
        name = "student_seq",
        sequenceName = "student_seq",
        allocationSize = 1
    )
    private Long id;

    private String name;
}
```

When to use it?

Use SEQUENCE when your database supports database sequences.

It is particularly common with:

- PostgreSQL
- Oracle

Example

Suppose we have:

student_sequence

The sequence generates:

1
2
3

4

...

Hibernate can obtain the next value from the sequence and use it as the Entity's ID.

Flow

```
graph TD
    A[Hibernate] --> B[Request next sequence value]
    B --> C[Database Sequence]
    C --> D[ID = 1]
    D --> E[Hibernate]
    E --> F[INSERT with ID = 1]
```

Database

Common examples:

- PostgreSQL
- Oracle

For example, a database sequence could be created as:

```
CREATE SEQUENCE student_seq
START WITH 1
INCREMENT BY 1;
```

4. GenerationType.TABLE

What is it?

TABLE uses a **separate database table** to generate and keep track of Primary Key values.

In simple words:

TABLE = Use a separate table for ID generation

Example:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    private Long id;

    private String name;
}
```

A generator table could conceptually look like:

generator_name	next_value
student	10

Hibernate can use the value **10** as the next ID and then update the table to **11**.

The next Entity can receive:

ID = 11

When to use it?

Use TABLE when:

- Table-based ID generation is specifically required.

- You cannot or don't want to rely on Identity or Sequence mechanisms.

However, it is **less commonly used in modern applications** because databases generally provide more efficient mechanisms such as Identity and Sequence.

Example

A more detailed configuration can use @TableGenerator:

```
@Entity
public class Student {

    @Id
    @GeneratedValue(
        strategy = GenerationType.TABLE,
        generator = "student_generator"
    )
    @TableGenerator(
        name = "student_generator",
        table = "id_generator",
        pkColumnName = "generator_name",
        valueColumnName = "next_value",
        pkColumnValue = "student",
        allocationSize = 1
    )
    private Long id;

    private String name;
}
```

Database

TABLE is not tied to one specific database because it uses a normal database table.

It can work with different databases, but it is usually not the first choice.

Comparison of the Four Strategies

Strategy	What does it do?	Who generates the ID?	When to use it?	Common Database
AUTO	Hibernate chooses the strategy	Hibernate/JPA Provider	When you don't need to control the strategy	Depends on DB
IDENTITY	Uses Identity/Auto Increment	Database	When the DB supports Auto Increment/Identity	MySQL, SQL Server
SEQUENCE	Uses a Database Sequence	Database Sequence	When the DB supports Sequences	PostgreSQL, Oracle
TABLE	Uses a separate table	Hibernate + Database Table	When table-based generation is required	Multiple databases

Final Summary

Strategy	Easy Meaning
AUTO	Hibernate decides
IDENTITY	Database Auto Increment
SEQUENCE	Database Sequence
TABLE	Separate Table generates IDs

The main idea is not just to memorize the four names. You should understand **who generates the ID and how**:

- **AUTO** → Hibernate chooses the strategy.
- **IDENTITY** → The database generates the ID using Identity/Auto Increment.
- **SEQUENCE** → A database Sequence generates the ID.
- **TABLE** → Hibernate uses a separate database table to manage ID generation.